

MAL Form Monitor — Complete User Guide

Version 1.0.0

This guide is written for someone who **doesn't have much programming experience**. You do not need to understand Python internals to use the program. Once everything is configured, the normal workflow is essentially:

 Google Form →  Google Sheet →  MAL Form Monitor →  Excel Report

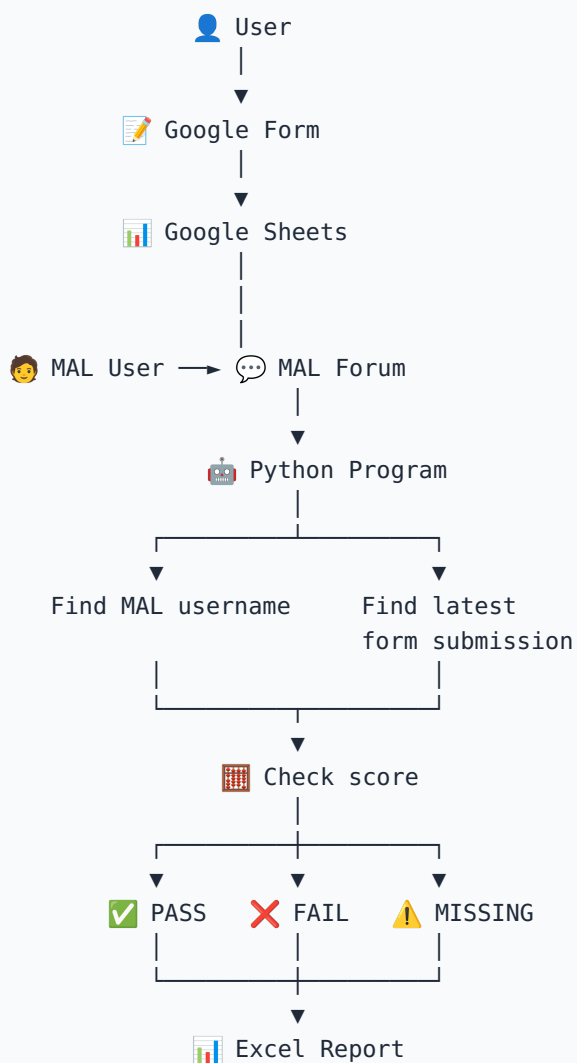
The program checks a MyAnimeList forum topic for people who have submitted the form, finds their latest form submission in Google Sheets, checks their score, and creates an Excel report showing who **Passed, Failed, or is Missing**.

Table of Contents

1.  What the program does
2.  What you need before starting
3.  Getting a MyAnimeList API Client ID
4.  Creating the Google Form
5.  Connecting the Form to Google Sheets
6.  Making the Google Sheet accessible without sign-in
7.  Finding your Spreadsheet ID
8.  Configuring the `.env` file
9.  Running the program
10.  Understanding the output files
11.  Recommended day-to-day workflow
12.  Testing the setup
13.  Common problems and fixes
14.  Tips for non-technical users
15.  Security and privacy
16.  Important limitations
17.  Final setup checklist

1. What the program does

The program is designed around a simple process:



The program currently expects a MAL forum reply containing something like:

```
Submitted the form!  
Username: test 1
```

The username is extracted automatically.

The program then searches the linked Google Sheet for that username.

It selects the **latest submission** based on the timestamp and reads its score.

The current configuration contains:

```
TOTAL_QUESTIONS=4  
PASSING_THRESHOLD=1
```

So, with the current settings:

Score ≥ 1 → PASS
Score < 1 → FAIL

The exact threshold can be changed through `.env`.

2. What you need before starting

You should have:

Required

-  A MyAnimeList account
-  A MAL API Client ID
-  A Google account
-  A Google Form
-  A Google Sheet containing the form responses
-  Python installed
-  The `mal-form-monitor` project
-  The required Python packages installed

The project should roughly look like:

```
mal-form-monitor/  
├──  
├── main.py  
├── config.py  
├── mal_client.py  
├── sheets_client.py  
├── state_manager.py  
├── report_writer.py  
├── logger_setup.py  
├── requirements.txt  
├── .env  
├──  
└── data/  
    ├── results.xlsx  
    ├── processed_state.json  
    └── monitor.log
```

You **do not normally need to edit the Python files**.

For normal usage, most configuration is done through `.env`.

3. Getting a MyAnimeList API Client ID

The program communicates with the MAL API using your **MAL API Client ID**.

The current program uses the `X-MAL-CLIENT-ID` HTTP header. The MAL API documentation you provided specifically documents Client-ID authentication for API requests that don't require a user login.

Step 1 — Log into MyAnimeList

Go to:

[MyAnimeList](#)

Log into the MAL account that you want to associate with your API application.

Step 2 — Open the MAL API configuration page

MAL provides an API configuration area where developers can create an API application/client.

Because MAL can change the layout of its website, the exact wording or location of buttons may change over time.

Look for the area related to:

```
API
API Configuration
Create an API application
```

Step 3 — Create an application

When creating an API application, MAL will ask for application information.

You may see fields such as:

```
App Name
Description
Homepage
Redirect URL
```

The exact fields depend on the current MAL developer interface.

For this particular project, you mainly need the resulting:

```
Client ID
```

 **Important**

You **do not need a user OAuth access token for the current version of this program.**

The program currently uses:

```
MAL_CLIENT_ID
```

rather than:

```
MAL_ACCESS_TOKEN
```

4. Creating the Google Form

The next part is your form.

You can create a Google Form through:

[Google Forms](#)

Google's documentation confirms that Forms can store responses in a linked Google Sheet.

Step 1 — Create the form

Create a new form.

For example:

```
MAL Quiz
```

You can add whatever questions you need.

For example:

```
Question 1  
Question 2  
Question 3  
Question 4
```



Username field — VERY IMPORTANT

The program needs to know which MAL user submitted the form.

Therefore your form should contain a field such as:

```
Username
```

For example:

MyAnimeList Username:
[_____]

The answer should be something like:

Trickedshark881

or:

test 1

depending on your system.

The username must match the MAL username

For example:

MAL forum:
Username: Trickedshark881

Google Sheet:
Trickedshark881

 Match.

But:

MAL forum:
Username: Trickedshark881

Google Sheet:
Tricked Shark881

may not match correctly.

The program does normalize capitalization and surrounding whitespace, but you should still keep usernames entered consistently.

5. **Connecting the Form to Google Sheets**

This is one of the most important parts.

Google Forms can automatically send responses to Google Sheets. Google's current instructions use the **Responses** → **Select response destination** workflow.

Step 1

Open your Google Form.

Click:

Responses

Step 2

Look for the Google Sheets option.

Depending on Google's current interface, you may see a Sheets icon or an option similar to:

View responses in Sheets

or:

Select response destination

Step 3

Choose:

Create a new spreadsheet

or:

Select existing spreadsheet

Google officially supports both options.

For a new project, I recommend:

Create a new spreadsheet

because it keeps the form and response data together automatically.

6. Making the Google Sheet accessible without sign-in

This is **extremely important**.

The program doesn't log into your Google account.

Instead, it reads the Google Sheet using Google's CSV export URL.

The current program constructs a URL like:

```
https://docs.google.com/spreadsheets/d/YOUR_SHEET_ID/gviz/tq?tqx=out:csv
```

and downloads the spreadsheet data.

Therefore the spreadsheet must be accessible to the program without requiring your Google account to log in.



Step-by-step: Make the Sheet accessible

Open your response spreadsheet.

Click:

Share

Find "General access"

You should see something similar to:

General access
Restricted

Click the dropdown.

Choose:

Anyone with the link

Then choose:

Viewer

So the final setting should look approximately like:

General access:
 Anyone with the link
 Viewer



DO NOT choose Editor

You only need:

Viewer

The Python program only needs to **read** the data.

It doesn't need permission to:

- edit the spreadsheet
- delete responses
- add responses
- change formulas

Therefore:

Anyone with the link → Viewer

is the appropriate setting.

Important privacy warning

Making the spreadsheet accessible to anyone with the link means the spreadsheet can be accessed by anyone who obtains that link.

Therefore, **do not put sensitive information into this spreadsheet.**

For example, avoid storing:

Passwords
Private addresses
Phone numbers
Private personal information
API secrets

If the spreadsheet only contains something like:

Timestamp | Username | Score

the privacy risk is much smaller, but you should still understand that the sheet is accessible through the link.

7. What is my Spreadsheet ID?

This is something that confuses many people.

Suppose your Google Sheet URL is:

<https://docs.google.com/spreadsheets/d/1ABCxyz1234567890abcdef/edit#gid=0>

The Spreadsheet ID is:

```
1ABCxyz1234567890abcdef
```

In other words:

```
https://docs.google.com/spreadsheets/d/
                        ↑
                    Spreadsheet ID
                        ↓
1ABCxyz1234567890abcdef
                        ↑
                    /edit#gid=0
```

✗ Don't include:

```
https://docs.google.com/spreadsheets/d/
```

✗ Don't include:

```
/edit#gid=0
```

✓ Only copy:

```
1ABCxyz1234567890abcdef
```

Example

If your Sheet URL is:

```
https://docs.google.com/spreadsheets/d/1aBcDeFgHiJkLmNoPqRsTuVwXyZ/edit#gid=0
```

then:

```
SHEET_ID=1aBcDeFgHiJkLmNoPqRsTuVwXyZ
```

That's what goes into `.env`.

8. Configuring the `.env` file

Your project uses an `.env` file for configuration.

The file contains settings such as:

```
MAL_CLIENT_ID=...
TOPIC_ID=...
SHEET_ID=...
TOTAL_QUESTIONS=4
PASSING_THRESHOLD=1
```

The project reads these values through `config.py`.



The problem with editing `.env` in Windows

Windows sometimes makes `.env` files inconvenient to edit because `.env` isn't a normal `.txt` file.

If you're uncomfortable editing it directly, you can use this simple method:

```
.env
↓
.env.txt
↓
Edit in Notepad
↓
.txt removed
↓
.env
```



Step 1 — Convert `.env` to `.txt`

Go to your project folder.

You should see:

```
.env
```

If Windows hides extensions, first enable them.

In File Explorer:

Click:

```
View → Show → File name extensions
```

Now Windows should display:

```
.env
```

instead of hiding the extension.

Rename it

Right-click:

`.env`

Choose:

Rename

Change it to:

`.env.txt`

Windows may warn:

Changing the file name extension might make the file unusable.

Click:

Yes



Step 2 — Open `.env.txt`

Right-click:

`.env.txt`

Choose:

Open with → Notepad

You should see something similar to:

```
MAL_CLIENT_ID=my client id
TOPIC_ID=forum topic id
SHEET_ID=g sheet id
TOTAL_QUESTIONS=4
PASSING_THRESHOLD=1

REPORT_PATH=data/results.xlsx
STATE_FILE=data/processed_state.json
LOG_FILE=data/monitor.log

AUDIT_ENABLED=true
SUMMARY_ENABLED=true
```

Step 3 — Replace the values

For example:

```
MAL_CLIENT_ID=123456789abcdef
TOPIC_ID=2280005
SHEET_ID=1aBcDeFgHiJkLmNoPqRsTuVwXyZ
TOTAL_QUESTIONS=4
PASSING_THRESHOLD=1

REPORT_PATH=data/results.xlsx
STATE_FILE=data/processed_state.json
LOG_FILE=data/monitor.log

AUDIT_ENABLED=true
SUMMARY_ENABLED=true
```

Important

Do **not** put quotation marks around values unless your configuration specifically requires them.

Use:

```
MAL_CLIENT_ID=123456
```

not:

```
MAL_CLIENT_ID="123456"
```

TOPIC_ID

Your topic ID comes from the MAL forum URL.

For example:

```
https://myanimelist.net/forum/?topicid=2280005
```

The topic ID is:

```
2280005
```

So:

```
TOPIC_ID=2280005
```

Your current `MALClient` uses this value to construct:

```
/v2/forum/topic/{topic_id}
```



TOTAL_QUESTIONS

If your quiz has four questions:

```
TOTAL_QUESTIONS=4
```

If it has ten:

```
TOTAL_QUESTIONS=10
```

This is currently a configuration value for the project.



PASSING_THRESHOLD

Suppose someone needs at least 3 points to pass:

```
PASSING_THRESHOLD=3
```

Then:

```
3 → PASS  
4 → PASS  
5 → PASS
```

while:

0 → FAIL
1 → FAIL
2 → FAIL

Step 4 — Save the file

In Notepad:

File → **Save**

or:

Ctrl + S

Step 5 — Convert it back to `.env`

Rename:

`.env.txt`

back to:

`.env`

Windows may again display a warning.

Click:

Yes



VERY IMPORTANT: Make sure it isn't `.env.txt`

This is one of the most common Windows problems.

You want:

`.env`

NOT:

`.env.txt`

NOT:

```
.env.env
```

NOT:

```
env
```

If you enabled **File name extensions**, this becomes much easier to verify.

9. Running the program

Once everything is configured, open **Command Prompt**.

Navigate to your project folder.

For example:

```
cd C:\Users\Admin\Desktop\mal-form-monitor
```

Your location may be different.



First: verify Python

Run:

```
python --version
```

You should get something like:

```
Python 3.13.14
```

The exact version can be different.



Install dependencies

If you haven't already installed the project's dependencies, run:

```
python -m pip install -r requirements.txt
```

You generally only need to do this once, or again if `requirements.txt` changes.



Start the monitor

Run:

```
python main.py
```

The program will start.

You should see console output indicating that the monitor has started and is processing the forum/form data.

What happens internally

The program performs roughly these steps:

1 Connects to MAL

```
MAL API
↓
Forum topic
```

The forum topic endpoint supports pagination through `limit` and `offset`.

2 Reads forum posts

It examines the returned posts.

3 Looks for the submission message

It searches for a pattern like:

```
Submitted the form!
Username: test 1
```

The parser is case-insensitive.

4 Extracts the username

For example:

```
Username: test 1
```

becomes:

```
test 1
```

5 Searches Google Sheets

The program downloads the spreadsheet as CSV and searches for that username.

6 Finds the latest submission

If someone submitted the form multiple times:

```
10:00 → 1/4  
11:00 → 2/4  
12:00 → 4/4
```

the program selects the newest timestamp.

7 Checks the score

For example:

```
4/4
```

is interpreted as:

```
4
```

while:

```
0/4
```

becomes:

```
0
```

8 Determines the result

For example:

```
Score = 4  
Threshold = 1  
  
4 >= 1  
  
→ PASS
```

9 Updates the Excel report

The program writes the result into:

```
data/results.xlsx
```

The report contains:

10. 📁 Understanding the output files

The `data` folder contains several files.



results.xlsx

This is your main result file.

Open it with:

- Microsoft Excel
- LibreOffice Calc
- another compatible spreadsheet program

It contains several worksheets.



Passed

People who met the passing threshold.



Failed

People who submitted a form but didn't meet the threshold.



Missing

People detected on MAL but for whom no matching Google Form submission could be found.



Summary

A quick overview such as:

Passed
Failed
Missing
Total Processed



Audit Log

The audit log contains information such as:

```
Reply Number
Forum User
Submitted Username
Score
Result
Form Timestamp
```

This is useful when you need to investigate why a particular person was classified a certain way.

processed_state.json

This file remembers which submissions have already been processed.

The purpose is to prevent the same submission from being processed repeatedly.

The current state manager tracks a processed timestamp for each normalized username.

Don't casually delete this file

If you delete it, the program may treat previously processed submissions as new again.

monitor.log

This is the program's log file.

Location:

```
data/monitor.log
```

If something goes wrong, this is one of the first files you should check.

11. **Recommended day-to-day workflow**

Once everything is configured, your normal routine can be extremely simple.

Step 1

People complete the Google Form.



Step 2

Google Forms automatically adds their responses to Google Sheets.



Step 3

People post their submission message on MAL.

For example:

```
Submitted the form!  
Username: test 2
```



Step 4

Run:

```
python main.py
```



Step 5

Open:

```
data/results.xlsx
```



Step 6

Check:

```
Passed  
Failed  
Missing
```

That's it. 🎉

12. 🧪 Testing your setup

Before relying on the program for real users, I strongly recommend doing a small test.

Create a test user/form submission.

For example:

```
Username: test 1  
Score: 4/4
```

Then create the corresponding MAL forum reply:

```
Submitted the form!  
Username: test 1
```

Run:

```
python main.py
```

Then open:

```
data/results.xlsx
```

You should find the user under:

```
Passed
```

if your threshold is:

```
PASSING_THRESHOLD=1
```



Test a failing submission

Create another test:

```
Username: test 2  
Score: 0/4
```

Then post:

```
Submitted the form!  
Username: test 2
```

Run:

```
python main.py
```

The user should appear under:

Failed

Test a missing user

Post on MAL:

```
Submitted the form!  
Username: someone_not_in_sheet
```

but don't submit the Google Form.

Run the program.

That person should appear under:

Missing

This is a useful test because it confirms the program can distinguish:

```
MAL submission exists  
+  
Google Form submission doesn't exist
```

13. Common problems and fixes

"Python is not recognized"

Try:

```
python --version
```

If that doesn't work, try:

```
py --version
```

If Python is installed but Windows is pointing to the wrong Python installation, that needs to be fixed before running the project.

ModuleNotFoundError

For example:

```
ModuleNotFoundError: No module named 'requests'
```

Run:

```
python -m pip install -r requirements.txt
```

Google Sheet can't be read

Check these **three things first**:

1. Is the Spreadsheet ID correct?

```
SHEET_ID=1abcDEF...
```

2. Is the sheet accessible without signing in?

Check:

```
Share
↓
General access
↓
Anyone with the link
↓
Viewer
```

3. Is the sheet actually the Form response spreadsheet?

Open the sheet and verify that the responses are appearing there.

"Missing" even though the person submitted the form

Check the username.

For example:

```
MAL:
test 2

Google Sheet:
test2
```

These are not necessarily the same username.

Also check the timestamp and the actual response row.

The program selects the **latest matching form submission**, so multiple submissions can affect which score is used.

✗ The program sees fewer MAL posts than the website

This is an important issue with the current version.

The program gets forum data through the MAL API rather than scraping the MAL webpage.

The API endpoint is:

```
https://api.myanimelist.net/v2/forum/topic/{topic_id}
```

If the MAL website displays a post that isn't returned by the API, **the Python parser cannot process that post because it never received it from MAL.**

This is different from a username parsing problem.

In other words:

```
MAL website
  ↓
Post exists visually
  ↓
API
  ↓
Post not returned
  ↓
Python
  ↓
Cannot detect it
```

The current project should therefore not be modified simply to "fix the parser" when the API itself hasn't returned the post.

This is particularly relevant to the investigation we've already been doing with topic `2280005`.

14. 💡 Tips for people with limited technical experience

★ Tip 1 — Don't edit Python files unless necessary

Most normal configuration should happen in:

```
.env
```

Avoid changing:

```
main.py
mal_client.py
sheets_client.py
state_manager.py
report_writer.py
```

unless you're intentionally modifying the program.

★ Tip 2 — Keep a backup

Before making major changes, copy the entire folder:

```
mal-form-monitor
```

and rename the copy:

```
mal-form-monitor-backup
```

This gives you an easy way to recover if something goes wrong.

★ Tip 3 — Don't rename project files

These files are imported by other files.

For example:

```
main.py
mal_client.py
sheets_client.py
```

Changing:

```
mal_client.py
```

to:

```
mal-api.py
```

could break imports.

★ Tip 4 — Keep the `.env` formatting simple

Use:

```
KEY=value
```

For example:

```
TOPIC_ID=2280005
```

Avoid accidentally doing:

```
TOPIC_ID = 2280005
```

or:

```
TOPIC_ID: 2280005
```

unless the configuration system is explicitly written to support that format.

★ Tip 5 — Don't put comments into `.env` unless you know they're supported

Keep it simple.

For example:

```
MAL_CLIENT_ID=abc123  
TOPIC_ID=2280005  
SHEET_ID=xyz456
```

★ Tip 6 — If something breaks, don't immediately delete files

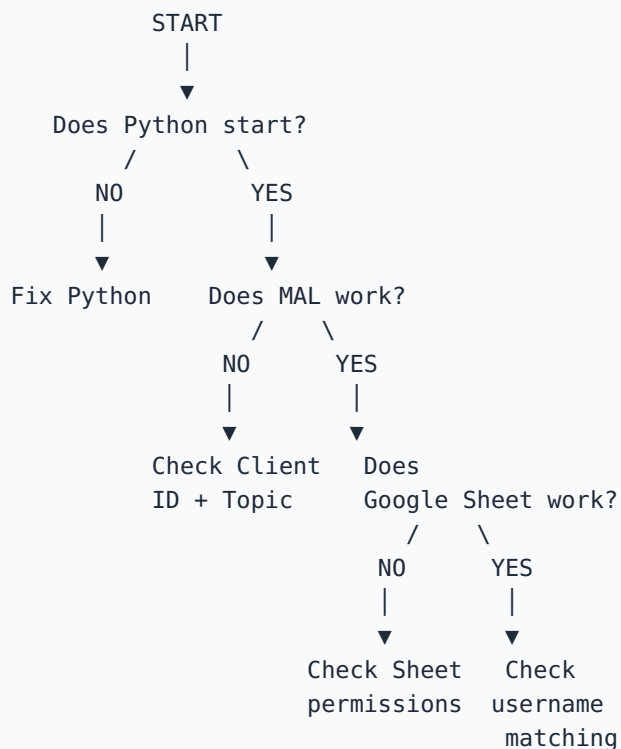
Instead, check:

1. Command Prompt error
2. data/monitor.log
3. `.env`
4. Google Sheet permissions
5. Spreadsheet ID
6. MAL topic ID

These six things solve a large portion of configuration problems.

A simple troubleshooting order

If the program doesn't work, follow this exact order:



This is much easier than randomly changing code.

15. Security and privacy

There are two important things to protect.

1. Your MAL Client ID

Treat your Client ID as configuration information.

Don't unnecessarily post your complete `.env` file publicly.

If you're asking someone for technical help, replace:

```
MAL_CLIENT_ID=123456789
```

with:

```
MAL_CLIENT_ID=YOUR_CLIENT_ID
```

2. Your `.env` file

Don't upload `.env` to GitHub if it contains private configuration.

Your `.gitignore` should ideally include:

```
.env  
data/
```

if you don't want those files committed.



Google Sheet privacy

Remember that the program requires the sheet to be accessible without your Google account.

Therefore:

```
Google Sheet  
  ↓  
Anyone with the link  
  ↓  
Viewer
```

means the contents are no longer restricted to your Google account.

Only put information into that sheet that you're comfortable making accessible to anyone who obtains its link.

16. 🧠 Understanding the "latest submission" behavior

This is particularly important if someone submits the Google Form more than once.

Suppose:

Timestamp	Username	Score
09:00	test 1	1/4
10:00	test 1	2/4
11:00	test 1	4/4

The program doesn't simply take the first row.

It identifies the matching username and selects the newest timestamp.

Therefore the result becomes:

```
test 1 → 4/4
```



Understanding duplicate prevention

The program also has a state system.

Suppose it processed:

```
test 1  
timestamp = 23/09/2026 11:00:00
```

It stores that information in:

```
data/processed_state.json
```

If you run the program again and nothing has changed, it can recognize that the submission has already been processed.

This is useful because you can run the program repeatedly without constantly reprocessing the same submission.

17. Recommended folder arrangement

I recommend keeping the project somewhere simple, such as:

```
C:\Users\Admin\Documents\mal-form-monitor\
```

rather than putting it somewhere complicated.

Inside:

```
mal-form-monitor  
|  
├── .env  
├── main.py  
├── config.py  
├── mal_client.py  
├── sheets_client.py  
├── state_manager.py  
├── report_writer.py  
├── logger_setup.py  
├── requirements.txt  
└── data  
    ├── results.xlsx  
    ├── processed_state.json  
    └── monitor.log
```

Complete setup example

Let's imagine your actual configuration is:

MAL Client ID:
ABC123XYZ

MAL Topic:
2280005

Google Sheet:
<https://docs.google.com/spreadsheets/d/1ABCxyz456DEF789/edit>

Your Spreadsheet ID is:

1ABCxyz456DEF789

Your `.env` would therefore contain:

```
MAL_CLIENT_ID=ABC123XYZ
TOPIC_ID=2280005
SHEET_ID=1ABCxyz456DEF789
TOTAL_QUESTIONS=4
PASSING_THRESHOLD=1

REPORT_PATH=data/results.xlsx
STATE_FILE=data/processed_state.json
LOG_FILE=data/monitor.log

AUDIT_ENABLED=true
SUMMARY_ENABLED=true
```

Then:

```
cd C:\Users\Admin\Documents\mal-form-monitor
```

and:

```
python main.py
```

The result should be generated at:

data\results.xlsx

The "Golden Rules"

If you remember nothing else, remember these:

Rule 1

Keep your `.env` private.

Rule 2

Your Google Sheet must be readable without a Google sign-in.

Anyone with the link → Viewer

Rule 3

Use the Spreadsheet ID, not the entire Google Sheets URL.

Rule 4

Make sure your Google Form responses are actually linked to the Sheet.

Rule 5

Make sure the Sheet has `Timestamp`, `Username`, and `Score` information.

Rule 6

Don't manually edit `processed_state.json` unless you know why you're doing it.

Rule 7

Don't change the Python files just because something isn't appearing in the report.

First determine whether the problem is:

```
MAL API
  ↓
Username extraction
  ↓
Google Sheet
  ↓
Username matching
  ↓
Score parsing
  ↓
State tracking
  ↓
Excel report
```

Rule 8

Always test with a few fake/test users before using the system for real submissions.

Final Setup Checklist

Before your first real run, verify every box:

MyAnimeList

- ☐ I have a MAL account.
- ☐ I created an API application.
- ☐ I have my **Client ID**.
- ☐ I know my forum **Topic ID**.

Google Forms

- ☐ My Google Form exists.
- ☐ It collects a MAL username.
- ☐ It collects/produces a score.
- ☐ Responses are enabled.

Google Sheets

- ☐ My Form is linked to a Google Sheet.
- ☐ Responses are appearing in the Sheet.
- ☐ The Sheet contains username information.
- ☐ The Sheet contains timestamps.
- ☐ The Sheet contains scores.
- ☐ General access is **Anyone with the link**.
- ☐ Permission is **Viewer**.
- ☐ I copied the correct Spreadsheet ID.

.env

- ☐ `MAL_CLIENT_ID` is correct.
- ☐ `TOPIC_ID` is correct.
- ☐ `SHEET_ID` is correct.
- ☐ `TOTAL_QUESTIONS` is correct.
- ☐ `PASSING_THRESHOLD` is correct.
- ☐ The file is actually named `.env`.
- ☐ It isn't accidentally `.env.txt`.

Python

- ☐ `python --version` works.

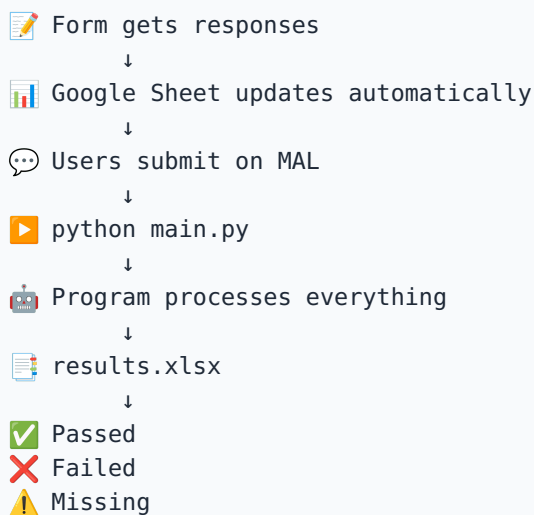
- ☐ Dependencies are installed.
- ☐ I am running the command from the project folder.

🟡 Final test

- ☐ A test user submitted the Google Form.
- ☐ The response appeared in Sheets.
- ☐ The same user posted the expected MAL message.
- ☐ `python main.py` runs.
- ☐ `results.xlsx` is generated.
- ☐ The user appears in the correct section.

🎉 Once everything is configured

Your regular process becomes:



Google Forms is designed to send responses to a linked spreadsheet automatically, so you don't need to manually export the responses each time.

One important note for this Version 1.0.0 guide: the MAL API visibility issue we've been investigating with topic `2280005` is **not something the user should try to fix through the setup steps above**. If the MAL website shows a post that the API doesn't return, the program cannot process that post because it never receives it from the API. That's a separate API/data-visibility issue from the normal installation and configuration process.